Information Management 1Notes

MODULE 1: INTRODUCTION TO DATABASE MANAGEMENT

What is a Database?

- Collection of data stored electronically
- Designed for easy storage and retrieval of information
- Organized, structured data repository

Database Management System (DBMS)

Definition: Software used to manage databases (e.g., MySQL, Oracle, SQL Server)

Core Functions:

1. **Data Storage Management** - Organize and store data efficiently
2. **Security Management** - Protect data from unauthorized access
3. **Backup & Recovery** - Prevent data loss
4. **Access Language & API** - Interface for data manipulation
5. **Data Dictionary Management** - Metadata storage
6. **Data Transformation** - Format conversions
7. **Multi-User Access Control** - Concurrent user management
8. **Data Integrity** - Ensure accuracy and consistency
9. **Database Communication** - Enable system interactions

DBMS: Pros & Cons

Advantages	Disadvantages
✓ Controls data redundancy	✗ High hardware/software costs
✓ Enables data sharing	✗ Large system size
✓ Easy maintenance	✗ System complexity
✓ Reduces processing time	✗ Higher impact if failure occurs
✓ Automated backups	
✓ Multiple user interfaces	

MODULE 2: DATABASE MODELS & TYPES

Database Types Classification

1. Centralized Database

- Single centralized storage location
- Users access from different locations
- Requires authentication
- **Example:** University central library system

2. Distributed Database

- Data spread across multiple locations
- Connected via communication links
- **Examples:** Apache Cassandra, HBase, Ignite

3. Relational Database (RDBMS)

- Stores data in **tables** (rows & columns)
- Uses **SQL** for operations
- Each table has a **primary key** for uniqueness
- Invented by E.F. Codd (1970)
- **Examples:** MySQL, Oracle, Microsoft SQL Server

Structure:

- **Row** = Tuple (single record)
- **Column** = Attribute (field)
- **Table** = Relation (collection of records)

4. NoSQL Database

- "Not Only SQL" - flexible data models
- Stores data beyond tabular format
- Built for modern, scalable applications
- **4 Subtypes:** Document, Key-Value, Column-family, Graph

5. Cloud Database

- Operates in virtual/cloud environment
- Provides SaaS, PaaS, IaaS services
- Scalable and accessible remotely

6. Object-Oriented Database

- Stores data as **objects** (like OOP)
- Supports complex data types
- Links to object-oriented programming languages

7. Hierarchical Database

- **Tree structure** (parent-child relationships)
- Single parent per child
- Top-down organization

8. Network Database

- **Graph structure** with interconnected nodes
- Multiple parents and children allowed

- More flexible than hierarchical

9. Personal Database

- Single-user system
- Stored on individual computers
- Designed for personal use

10. Operational Database

- Handles **real-time** daily transactions
- Supports day-to-day business operations
- Frequent updates and queries

11. Enterprise Database

- Manages **massive data volumes**
- Supports simultaneous multi-user access
- Used by large organizations

MODULE 3: INTRODUCTION TO SQL

SQL Fundamentals

SQL = Structured Query Language for RDBMS

Pattern Matching: LIKE Operator

Syntax:

```
SELECT column1, column2
FROM table_name
WHERE columnN LIKE 'pattern';
```

Two Wildcards:

- **%** - Represents zero, one, or multiple characters

- _ - Represents exactly ONE character

LIKE Pattern Examples

Pattern	Matches	Example
`'a%'`	Starts with 'a'	Alfred, Anna, Apple
`'%a'`	Ends with 'a'	Maria, Anna, Pizza
`'%or%'`	Contains 'or' anywhere	Toronto, Morning, Tutor
`'_r%'`	'r' in 2nd position	Brown, Trial, Great
`'a_%'`	Starts with 'a', min 2 chars	Ab, Ann, Apple
`'a__%'`	Starts with 'a', min 3 chars	Abc, Anna, Apple
`'a%o'`	Starts with 'a', ends with 'o'	Antonio, Amigo
`NOT LIKE 'a%'`	Does NOT start with 'a'	Brown, City, David

Advanced Wildcards

Character Sets:

```
-- Starts with b, s, OR p
WHERE City LIKE '[bsp]%'

-- Starts with a, b, or c (range)
WHERE City LIKE '[a-c]%'

-- Does NOT start with b, s, or p
WHERE City LIKE '[!bsp]%'
-- OR
WHERE City NOT LIKE '[bsp]%'
```

Wildcard Use Cases

```
-- City starting with 'ber'
WHERE City LIKE 'ber%'

-- City containing 'es'
```

```
WHERE City LIKE '%es%'

-- City: any char + 'ondon' (London, Condon, etc.)
WHERE City LIKE '_ondon'

-- Pattern: L + any + n + any + on
WHERE City LIKE 'L_n_on'  -- Matches: London
```

MODULE 4: SQL IN OPERATOR

IN Operator

Purpose: Shorthand for multiple OR conditions

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1, value2, ...);
```

Examples:

```
-- Select customers from specific countries
WHERE Country IN ('Germany', 'France', 'UK')

-- Exclude specific countries
WHERE Country NOT IN ('Germany', 'France', 'UK')

-- Using subquery
WHERE Country IN (SELECT Country FROM Suppliers)
```

When to Use:

- ✓ Multiple exact value matches needed
- ✓ Cleaner than chaining ORs
- ✓ Can use with subqueries

MODULE 5: SQL BETWEEN OPERATOR

BETWEEN Operator

Purpose: Select values within a range (INCLUSIVE)

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Data Types Supported:

- Numbers
- Text (alphabetical)
- Dates

Numeric BETWEEN

```
-- Price from 10 to 20 (inclusive)
WHERE Price BETWEEN 10 AND 20

-- Outside the range
WHERE Price NOT BETWEEN 10 AND 20

-- Combined with NOT IN
WHERE Price BETWEEN 10 AND 20
  AND CategoryID NOT IN (1,2,3)
```

Text BETWEEN

```
-- Alphabetical range (inclusive)
WHERE ProductName BETWEEN 'Carnarvon Tigers' AND 'Filo Mix'
ORDER BY ProductName;
```

```
-- NOT BETWEEN text
WHERE ProductName NOT BETWEEN 'Carnarvon Tigers' AND 'Mozzare
lla di Giovanni'
```

Date BETWEEN

```
-- Date range format 1 (Access)
WHERE OrderDate BETWEEN #07/01/1996# AND #07/31/1996#

-- Date range format 2 (Standard SQL)
WHERE OrderDate BETWEEN '1996-07-01' AND '1996-07-31'
```

MODULE 6: SQL ALIASES

What are Aliases?

- Temporary names for tables or columns
- Exist only during query execution
- Make queries more readable
- Required when using multiple tables with same column names

Column Aliases

Syntax:

```
-- Using AS keyword (recommended)
SELECT column_name AS alias_name
FROM table_name;

-- Without AS keyword
SELECT column_name alias_name
FROM table_name;
```

Examples:

```
-- Single column alias
SELECT CustomerName AS Customer
FROM Customers;

-- Multiple column aliases
SELECT CustomerID AS ID, CustomerName AS Customer
FROM Customers;

-- Alias with spaces (requires quotes)
SELECT CustomerName AS "Customer Name"
FROM Customers;

-- Combining columns with alias
SELECT FirstName + ' ' + LastName AS FullName
FROM Employees;

-- MySQL concatenation
SELECT CONCAT(FirstName, ' ', LastName) AS FullName
FROM Employees;

-- Alias with calculations
SELECT Price * Quantity AS TotalCost
FROM OrderDetails;
```

Table Aliases

Syntax:

```
SELECT column_name
FROM table_name AS alias_name;
```

Examples:

```
-- Simple table alias
SELECT c.CustomerName, c.City
FROM Customers AS c;

-- Multiple table aliases
SELECT o.OrderID, c.CustomerName
FROM Orders AS o, Customers AS c
WHERE o.CustomerID = c.CustomerID;

-- Shorter aliases for readability
SELECT p.ProductName, s.CompanyName
FROM Products AS p
INNER JOIN Suppliers AS s
ON p.SupplierID = s.SupplierID;
```

When to Use Aliases

✓ **Use aliases when:**

- Joining multiple tables
- Column names are too long
- Combining multiple columns
- Performing calculations
- Improving query readability
- Same column name exists in multiple tables

Alias Best Practices

```
-- ✓ GOOD: Clear and descriptive
SELECT
    e.EmployeeID AS EmpID,
    e.FirstName + ' ' + e.LastName AS FullName,
    d.DepartmentName AS Department
FROM Employees AS e
```

```
INNER JOIN Departments AS d
ON e.DepartmentID = d.DepartmentID;

-- ✗ AVOID: Confusing single letters without context
SELECT a.x, b.y
FROM tab1 AS a, tab2 AS b;
```

Common Alias Patterns

```
-- Pattern 1: Aggregation with alias
SELECT
    COUNT(*) AS TotalOrders,
    SUM(Amount) AS TotalRevenue,
    AVG(Amount) AS AverageOrderValue
FROM Orders;

-- Pattern 2: CASE statement with alias
SELECT
    ProductName,
    CASE
        WHEN Price < 10 THEN 'Cheap'
        WHEN Price BETWEEN 10 AND 50 THEN 'Moderate'
        ELSE 'Expensive'
    END AS PriceCategory
FROM Products;

-- Pattern 3: Subquery with alias
SELECT
    CustomerName,
    TotalOrders
FROM (
    SELECT
        CustomerID,
        COUNT(*) AS TotalOrders
    FROM Orders
```

```
    GROUP BY CustomerID
) AS OrderSummary
INNER JOIN Customers AS c
ON OrderSummary.CustomerID = c.CustomerID;
```

MODULE 7: SQL JOINS

What are SQL Joins?

- Combine rows from two or more tables
- Based on related columns between them
- Essential for working with relational databases

Types of Joins

```
┌─────────────────────────────────────────┐
│         SQL JOIN TYPES                  │
├─────────────────────────────────────────┤
│  1. INNER JOIN (most common)            │
│  2. LEFT (OUTER) JOIN                   │
│  3. RIGHT (OUTER) JOIN                  │
│  4. FULL (OUTER) JOIN                   │
│  5. CROSS JOIN                          │
│  6. SELF JOIN                           │
└─────────────────────────────────────────┘
```

---

1. INNER JOIN

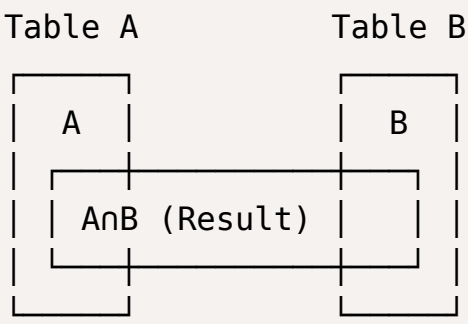
Definition: Returns records that have matching values in BOTH tables

Syntax:

```
SELECT column_name(s)
FROM table1
```

```
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

Visual Representation:



Example:

```
-- Get orders with customer information
SELECT
    Orders.OrderID,
    Customers.CustomerName,
    Orders.OrderDate
FROM Orders
INNER JOIN Customers
ON Orders.CustomerID = Customers.CustomerID;
```

Result: Only orders that have a matching customer

Three-Table Inner Join:

```
SELECT
    o.OrderID,
    c.CustomerName,
    s.ShipperName
FROM Orders AS o
INNER JOIN Customers AS c ON o.CustomerID = c.CustomerID
INNER JOIN Shippers AS s ON o.ShipperID = s.ShipperID;
```

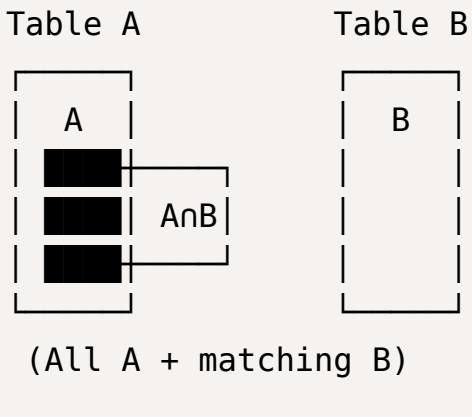
2. LEFT JOIN (LEFT OUTER JOIN)

Definition: Returns ALL records from left table + matched records from right table

Syntax:

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

Visual Representation:



Example:

```
-- Get ALL customers and their orders (if any)
SELECT
    c.CustomerName,
    o.OrderID
FROM Customers AS c
LEFT JOIN Orders AS o
ON c.CustomerID = o.CustomerID
ORDER BY c.CustomerName;
```

Result:

- All customers shown
- Customers WITHOUT orders show NULL for OrderID

Finding Records with NO Match:

```
-- Customers who have NEVER placed an order
SELECT
    c.CustomerName
FROM Customers AS c
LEFT JOIN Orders AS o
ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```

---

3. RIGHT JOIN (RIGHT OUTER JOIN)

Definition: Returns ALL records from right table + matched records from left table

Syntax:

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

Visual Representation:

```
Table A          Table B
┌─────┐          ┌─────┐
│  A  │          │  B  │
│     │     ┌────┼──■■ │
│     │ A∩B │    │■■■■ │
│     │     └────┼──■■ │
└─────┘          └─────┘
 (All B + matching A)
```

Example:

```
-- Get ALL orders and customer info (if available)
SELECT
```

```
    o.OrderID,
    c.CustomerName
FROM Orders AS o
RIGHT JOIN Customers AS c
ON o.CustomerID = c.CustomerID;
```

Note: RIGHT JOIN is less common - most developers prefer LEFT JOIN and swap table order

Equivalent Queries:

```
-- These produce the same result:
SELECT * FROM Orders RIGHT JOIN Customers ON Orders.CustomerI
D = Customers.CustomerID;
SELECT * FROM Customers LEFT JOIN Orders ON Customers.Custome
rID = Orders.CustomerID;
```

---

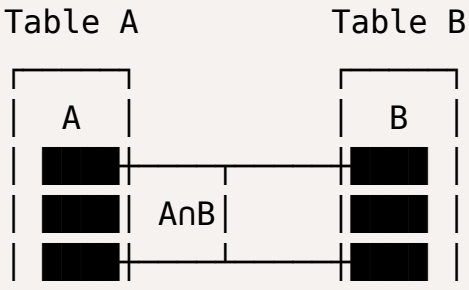
4. FULL OUTER JOIN

Definition: Returns ALL records when there's a match in either left OR right table

Syntax:

```
SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name;
```

Visual Representation:



```
└──────┘           └──────┘
 (Everything from both tables)
```

Example:

```
-- Get ALL customers and ALL orders
SELECT
    c.CustomerName,
    o.OrderID
FROM Customers AS c
FULL OUTER JOIN Orders AS o
ON c.CustomerID = o.CustomerID
ORDER BY c.CustomerName;
```

Result:

- Customers without orders (OrderID = NULL)
- Orders without customers (CustomerName = NULL)
- Matched customer-order pairs

Finding Mismatches:

```
-- Records that DON'T match in either direction
SELECT
    c.CustomerName,
    o.OrderID
FROM Customers AS c
FULL OUTER JOIN Orders AS o
ON c.CustomerID = o.CustomerID
WHERE c.CustomerID IS NULL OR o.CustomerID IS NULL;
```

Note: Not supported in MySQL - use UNION of LEFT and RIGHT JOIN instead

---

5. CROSS JOIN

Definition: Returns the Cartesian product (ALL possible combinations)

Syntax:

```
SELECT column_name(s)
FROM table1
CROSS JOIN table2;
```

Visual Representation:

```
Table A (3 rows)  ×  Table B (2 rows)  =  6 rows
     A                    B                  Result
┌──────────┐          ┌──────────┐          ┌──────────────┐
│   A1     │          │   B1     │          │ A1 - B1      │
│   A2     │    ×     │   B2     │    =     │ A1 - B2      │
│   A3     │          └──────────┘          │ A2 - B1      │
└──────────┘                                │ A2 - B2      │
                                            │ A3 - B1      │
                                            │ A3 - B2      │
                                            └──────────────┘
```

Example:

```
-- All possible combinations of products and sizes
SELECT
    p.ProductName,
    s.SizeName
FROM Products AS p
CROSS JOIN Sizes AS s;
```

Alternative Syntax (older):

```
-- Same as CROSS JOIN
SELECT p.ProductName, s.SizeName
FROM Products AS p, Sizes AS s;
```

Use Cases:

- Generating test data
- Creating all possible combinations (sizes, colors, etc.)
- Calendar tables (dates × time slots)

⚠️ **Warning:** Result size = (rows in table1) × (rows in table2)

- 1000 × 1000 = 1,000,000 rows!

---

6. SELF JOIN

Definition: Join a table to itself

Syntax:

```
SELECT column_name(s)
FROM table1 AS t1
INNER JOIN table1 AS t2
ON t1.column_name = t2.column_name;
```

Example 1: Employee-Manager Relationship

```
-- Employee table with ManagerID column
SELECT
    e.EmployeeName AS Employee,
    m.EmployeeName AS Manager
FROM Employees AS e
INNER JOIN Employees AS m
ON e.ManagerID = m.EmployeeID;
```

Example 2: Finding Pairs

```
-- Customers from the same city
SELECT
    c1.CustomerName AS Customer1,
    c2.CustomerName AS Customer2,
    c1.City
FROM Customers AS c1
```

```
INNER JOIN Customers AS c2
ON c1.City = c2.City
WHERE c1.CustomerID < c2.CustomerID  -- Avoid duplicates
ORDER BY c1.City;
```

Example 3: Hierarchical Data

```
-- Categories and their parent categories
SELECT
    child.CategoryName AS Subcategory,
    parent.CategoryName AS ParentCategory
FROM Categories AS child
LEFT JOIN Categories AS parent
ON child.ParentCategoryID = parent.CategoryID;
```

Use Cases:

- Organizational hierarchies
- Comparing rows within same table
- Finding duplicates or relationships
- Tree structures

JOIN COMPARISON TABLE

JOIN Type	Returns	Use When
INNER	Only matching records	Need data that exists in BOTH tables
LEFT	All from left + matches	Need all records from main table
RIGHT	All from right + matches	Need all records from secondary table
FULL OUTER	All from both tables	Need to see all data including mismatches
CROSS	All combinations	Need Cartesian product
SELF	Table joined to itself	Need to compare rows within same table

---

COMPREHENSIVE JOIN EXAMPLES

Example 1: Simple E-commerce Query

```
-- Get order details with product and customer info
SELECT
    o.OrderID,
    c.CustomerName,
    p.ProductName,
    od.Quantity,
    od.Quantity * p.Price AS TotalPrice
FROM Orders AS o
INNER JOIN Customers AS c ON o.CustomerID = c.CustomerID
INNER JOIN OrderDetails AS od ON o.OrderID = od.OrderID
INNER JOIN Products AS p ON od.ProductID = p.ProductID
WHERE o.OrderDate >= '2024-01-01';
```

Example 2: Finding Gaps

```
-- Products that have NEVER been ordered
SELECT p.ProductName
FROM Products AS p
LEFT JOIN OrderDetails AS od
ON p.ProductID = od.ProductID
WHERE od.ProductID IS NULL;
```

Example 3: Multiple Conditions

```
-- Advanced join with multiple conditions
SELECT
    e.EmployeeName,
    d.DepartmentName,
    s.SalaryAmount
FROM Employees AS e
INNER JOIN Departments AS d
    ON e.DepartmentID = d.DepartmentID
LEFT JOIN Salaries AS s
```

```
    ON e.EmployeeID = s.EmployeeID
    AND s.Year = 2024
WHERE d.Location = 'New York';
```

Example 4: Aggregation with Joins

```
-- Customer order summary
SELECT
    c.CustomerName,
    COUNT(o.OrderID) AS TotalOrders,
    SUM(o.TotalAmount) AS TotalSpent,
    AVG(o.TotalAmount) AS AvgOrderValue
FROM Customers AS c
LEFT JOIN Orders AS o
ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.CustomerName
HAVING COUNT(o.OrderID) > 0
ORDER BY TotalSpent DESC;
```

Example 5: Subquery in JOIN

```
-- Customers who ordered above-average amounts
SELECT
    c.CustomerName,
    o.TotalAmount
FROM Customers AS c
INNER JOIN Orders AS o
ON c.CustomerID = o.CustomerID
WHERE o.TotalAmount > (
    SELECT AVG(TotalAmount)
    FROM Orders
);
```

JOIN PERFORMANCE TIPS

Optimization Best Practices

✓ **Use indexes on JOIN columns**

```
CREATE INDEX idx_customer_id ON Orders(CustomerID);
```

✓ **Filter early with WHERE**

```
-- GOOD: Filter before joining
SELECT o.*, c.*
FROM Orders AS o
INNER JOIN Customers AS c ON o.CustomerID = c.CustomerID
WHERE o.OrderDate >= '2024-01-01';  -- Filter reduces dataset

-- AVOID: Filtering after large join
```

✓ **Select only needed columns**

```
-- GOOD: Specific columns
SELECT c.CustomerName, o.OrderID

-- AVOID: SELECT *
```

✓ **Use appropriate JOIN type**

```
-- If you need ALL left records, use LEFT JOIN
-- Don't use FULL OUTER JOIN when LEFT JOIN suffices
```

✓ **Consider JOIN order**

```
-- Start with smallest table
FROM SmallTable AS s
INNER JOIN LargeTable AS l ON s.ID = l.ID
```

COMMON JOIN MISTAKES & SOLUTIONS

Mistake 1: Missing JOIN Condition

```
-- ❌ WRONG: Accidental CROSS JOIN
SELECT *
FROM Customers, Orders;

-- ✅ CORRECT: Specify JOIN condition
SELECT *
FROM Customers AS c
INNER JOIN Orders AS o ON c.CustomerID = o.CustomerID;
```

Mistake 2: Ambiguous Column Names

```
-- ❌ WRONG: Which CustomerID?
SELECT CustomerID, OrderID
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerI
D;

-- ✅ CORRECT: Always prefix
SELECT
    c.CustomerID,
    o.OrderID
FROM Customers AS c
INNER JOIN Orders AS o ON c.CustomerID = o.CustomerID;
```

Mistake 3: Wrong JOIN Type

```
-- ❌ WRONG: INNER JOIN misses customers without orders
SELECT c.CustomerName, COUNT(o.OrderID)
FROM Customers AS c
INNER JOIN Orders AS o ON c.CustomerID = o.CustomerID
```

```
GROUP BY c.CustomerName;

-- ✅ CORRECT: LEFT JOIN includes all customers
SELECT c.CustomerName, COUNT(o.OrderID)
FROM Customers AS c
LEFT JOIN Orders AS o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerName;
```

Mistake 4: Forgetting NULL Handling

```
-- ❌ WRONG: Might miss records
WHERE o.ShipDate = '2024-01-01'

-- ✅ CORRECT: Handle NULLs
WHERE o.ShipDate = '2024-01-01' OR o.ShipDate IS NULL
```

ADVANCED JOIN TECHNIQUES

1. Using USING Clause

```
-- When column names are identical
SELECT *
FROM Orders
INNER JOIN Customers USING (CustomerID);

-- Equivalent to:
SELECT *
FROM Orders AS o
INNER JOIN Customers AS c ON o.CustomerID = c.CustomerID;
```

2. Natural Join (Use with Caution)

```
-- Automatically joins on ALL matching column names
SELECT *
FROM Orders
NATURAL JOIN Customers;

-- ⚠ Risky: Might join on unintended columns
```

3. Lateral Joins (PostgreSQL, Oracle)

```
-- For each customer, get their 3 most recent orders
SELECT c.CustomerName, recent.*
FROM Customers AS c
CROSS JOIN LATERAL (
    SELECT OrderID, OrderDate
    FROM Orders
    WHERE CustomerID = c.CustomerID
    ORDER BY OrderDate DESC
    LIMIT 3
) AS recent;
```

4. Join with CASE

```
SELECT
    p.ProductName,
    CASE
        WHEN s.StockQuantity > 100 THEN 'In Stock'
        WHEN s.StockQuantity BETWEEN 1 AND 100 THEN 'Low Stoc
k'
        ELSE 'Out of Stock'
    END AS StockStatus
FROM Products AS p
LEFT JOIN Stock AS s ON p.ProductID = s.ProductID;
```

QUICK DECISION FLOWCHART

```
Need data from multiple tables?
    │
    ├─ YES → Need ALL records from one table?
    │         │
    │         ├─ From LEFT table → LEFT JOIN
    │         ├─ From RIGHT table → RIGHT JOIN
    │         ├─ From BOTH tables → FULL OUTER JOIN
    │         └─ Only matching records → INNER JOIN
    │
    ├─ Need ALL combinations? → CROSS JOIN
    │
    └─ Comparing within same table? → SELF JOIN
```

PRACTICE EXERCISES

Exercise 1: Basic JOIN

Write a query to display all order IDs with customer names and order dates.

<details>
<summary>Solution</summary>

```
SELECT
    o.OrderID,
    c.CustomerName,
    o.OrderDate
FROM Orders AS o
INNER JOIN Customers AS c
ON o.CustomerID = c.CustomerID;
```

</details>

Exercise 2: LEFT JOIN with NULL

Find all customers who have never placed an order.

<details>
<summary>Solution</summary>

```
SELECT c.CustomerName
FROM Customers AS c
LEFT JOIN Orders AS o
ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```

</details>

Exercise 3: Multiple JOINs

Display order details with customer name, product name, and shipper name.

<details>
<summary>Solution</summary>

```
SELECT
    o.OrderID,
    c.CustomerName,
    p.ProductName,
    s.ShipperName
FROM Orders AS o
INNER JOIN Customers AS c ON o.CustomerID = c.CustomerID
INNER JOIN OrderDetails AS od ON o.OrderID = od.OrderID
INNER JOIN Products AS p ON od.ProductID = p.ProductID
INNER JOIN Shippers AS s ON o.ShipperID = s.ShipperID;
```

</details>

Exercise 4: SELF JOIN

Find all employees and their managers from an Employees table.

<details>
<summary>Solution</summary>

```
SELECT
    e.EmployeeName AS Employee,
    m.EmployeeName AS Manager
FROM Employees AS e
LEFT JOIN Employees AS m
ON e.ManagerID = m.EmployeeID;
```

</details>

SUMMARY :

Remember These Core Concepts:

1. **Aliases** make queries readable and are essential with JOINs
2. **INNER JOIN** = intersection (only matches)
3. **LEFT/RIGHT JOIN** = include all from one side
4. **FULL OUTER JOIN** = union (everything)
5. **CROSS JOIN** = Cartesian product (all combinations)
6. **SELF JOIN** = table joining itself
7. Always specify JOIN conditions to avoid accidental CROSS JOINs
8. Use appropriate JOIN type based on your data needs
9. Filter with WHERE to improve performance
10. Handle NULLs properly in outer joins

SQL Query Order of Execution:

```
FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
```

APPENDIX: SQL OPERATOR SUMMARY

Operator	Purpose	Example
`LIKE`	Pattern matching	`WHERE Name LIKE 'A%'`
`IN`	Multiple values	`WHERE Country IN ('US', 'UK')`
`BETWEEN`	Range (inclusive)	`WHERE Age BETWEEN 18 AND 65`
`=`	Exact match	`WHERE ID = 5`
`!=` or `<>`	Not equal	`WHERE Status != 'Inactive'`
`IS NULL`	Check for NULL	`WHERE Email IS NULL`
`IS NOT NULL`	Check for non-NULL	`WHERE Phone IS NOT NULL`